

Routing Protocols

Outline

11.1 Introduction

*11.2 Intra- and Inter-Domain
Routing*

11.3 Distance Vector Routing

11.4 RIP

11.5 Link State Routing

11.6 OSPF

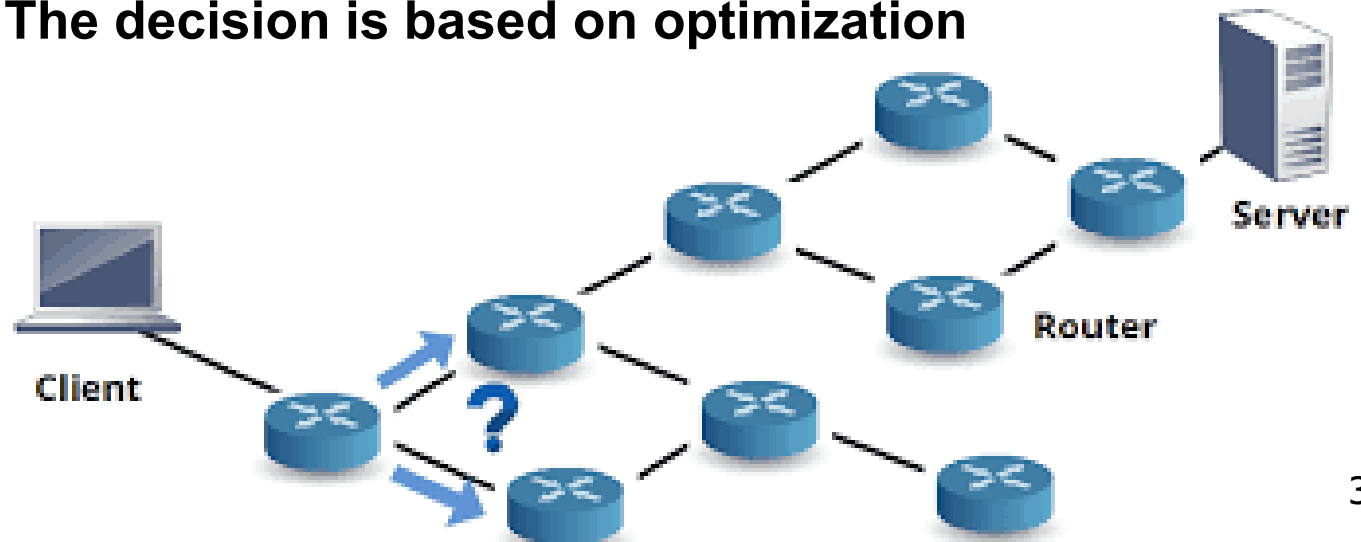
INTRODUCTION

An internet is a combination of networks connected by routers.

When a datagram goes from a source to a destination, it will probably pass through many routers until it reaches the router attached to the destination network.

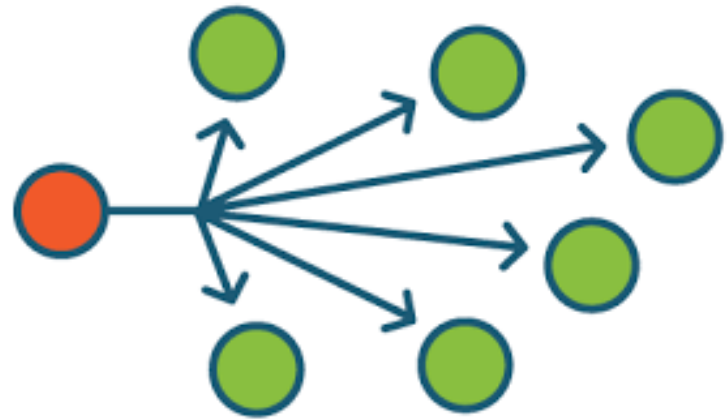
There is need of decision making required, to which network the packet should pass?

The decision is based on optimization



■ Flooding

- A packet is sent by a source node to every one of its neighbors
- At each node, an incoming packet is forwarded on all outgoing link except for the link on which it arrived



■ Properties

- All possible routes btw source and destination are tried
- At least one copy of the packet to arrive at the destination will have a minimum-hop route
- All nodes connected to the source node are visited

Static Versus Dynamic Routing



- Static –

one which created by manual entries

- Dynamic –

one which is updated automatically when there is a change somewhere in the internet

INTER- AND INTRA-DOMAIN ROUTING

Today, an internet can be so large that **one routing protocol cannot handle the task of updating** the routing tables of all routers.

For this reason, an internet is divided into autonomous systems.

An **autonomous system (AS)** is a group of networks and routers under the authority of a single administration.

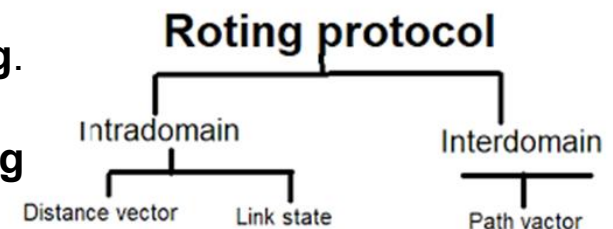
The Internet Assigned Numbers Authority (IANA) is responsible for managing and allocating Autonomous System Numbers (ASNs) across the internet.

While IANA oversees the overall ASN system, the actual assignment of ASNs to individual Autonomous Systems (AS) is delegated to Regional Internet Registries (RIRs).

These RIRs, manage the allocation of ASNs within their respective geographic regions

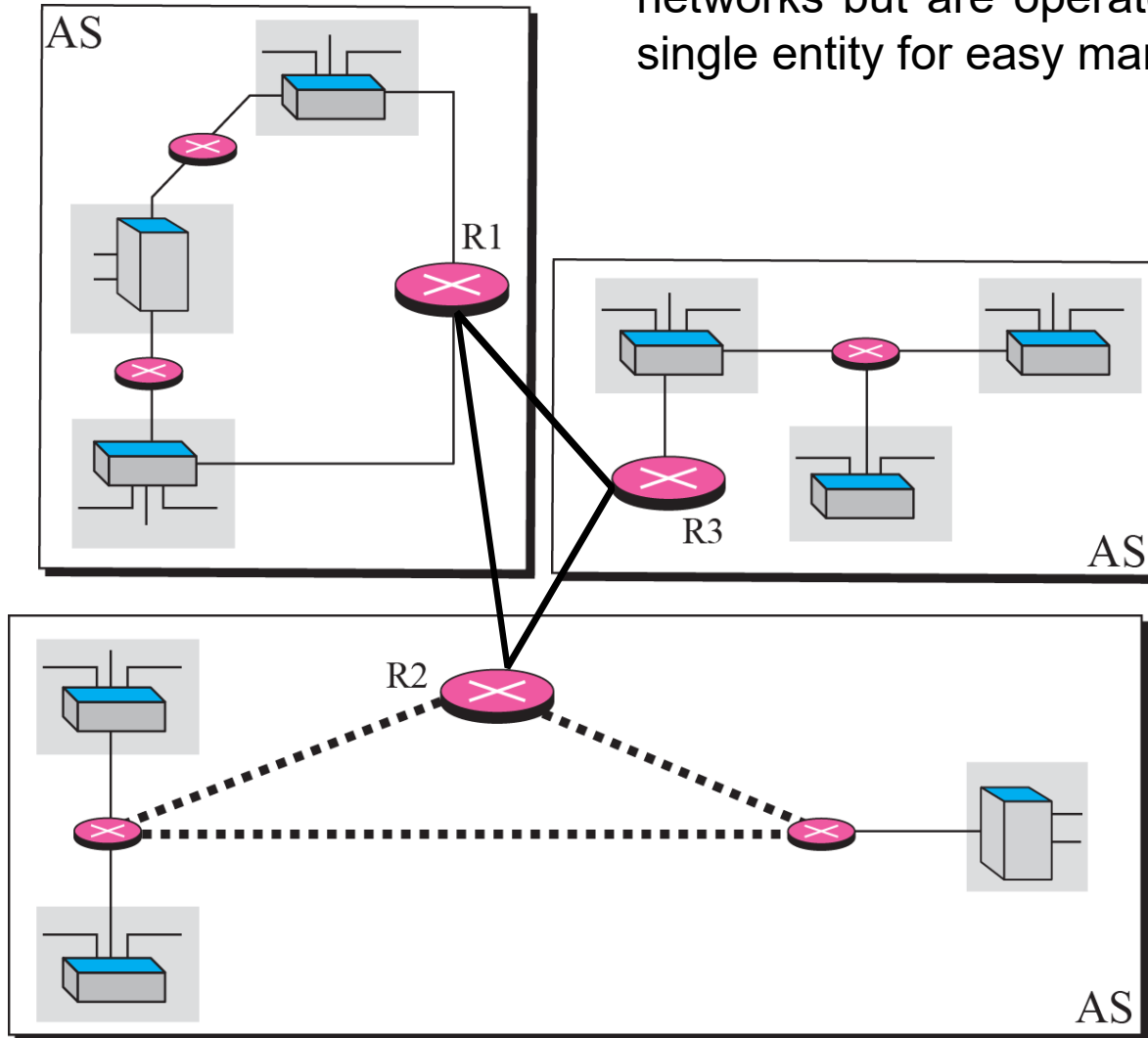
Routing inside an autonomous system is **intra-domain routing**.

Routing between autonomous systems is **inter-domain routing**

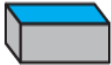


Autonomous systems

These systems are made up of many different networks but are operated under the umbrella of a single entity for easy management



Legend

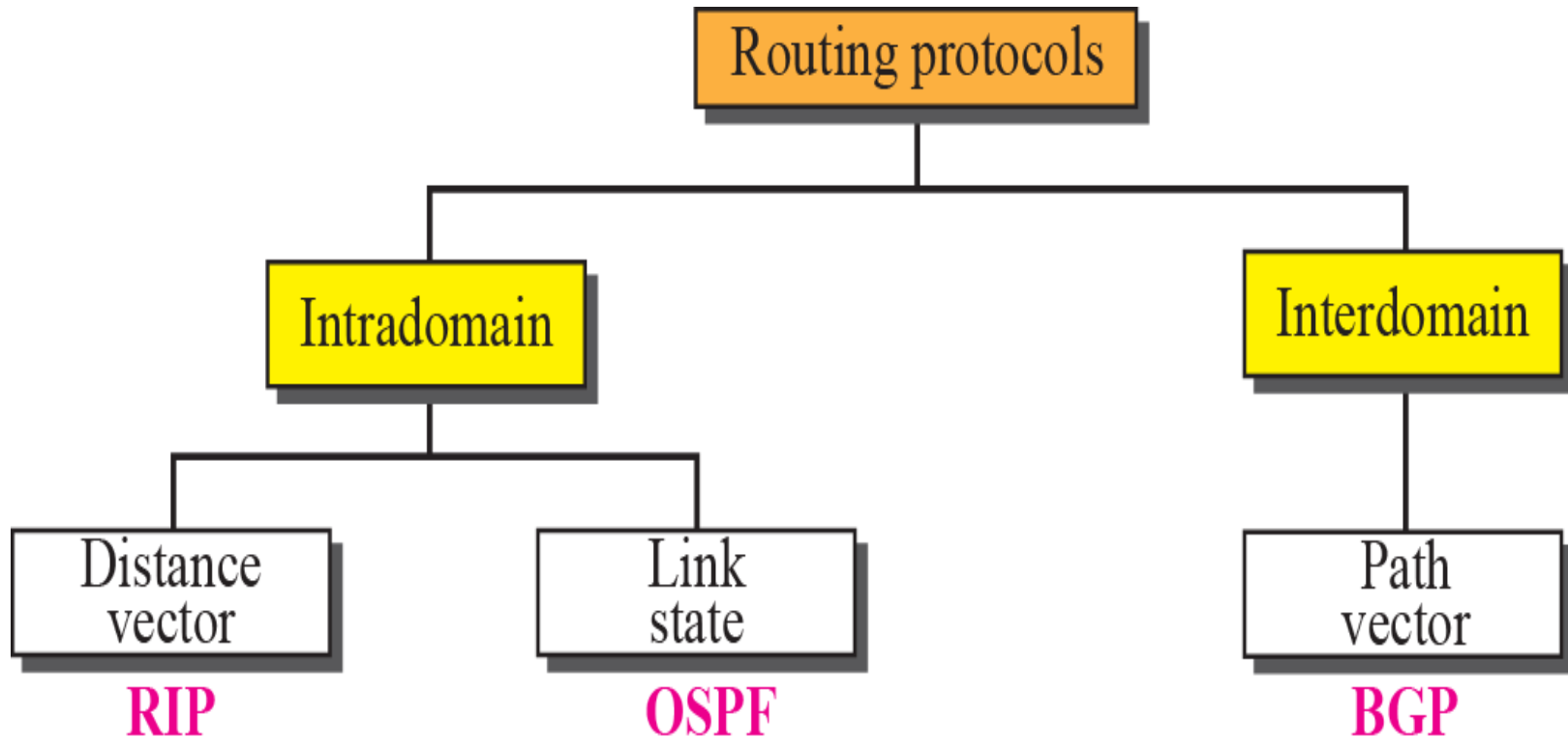
- AS Autonomous System
-  Ethernet switch
- Point-to-point WAN
- Inter-system connection



Popular routing protocols

interior

exterior



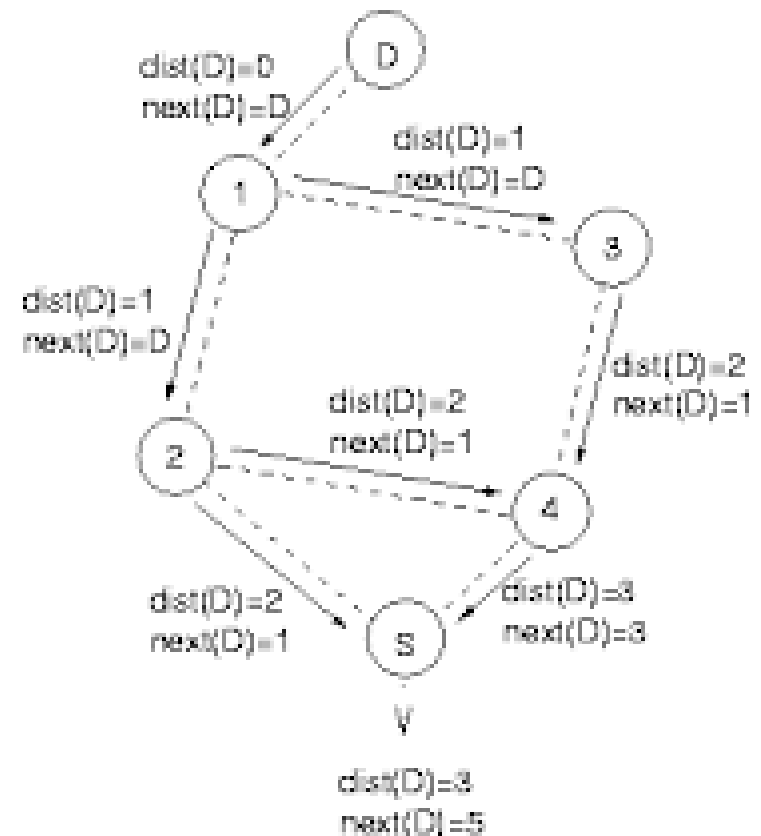
DISTANCE VECTOR ROUTING

This method sees an autonomous system (AS) with all routers and networks, as a graph,

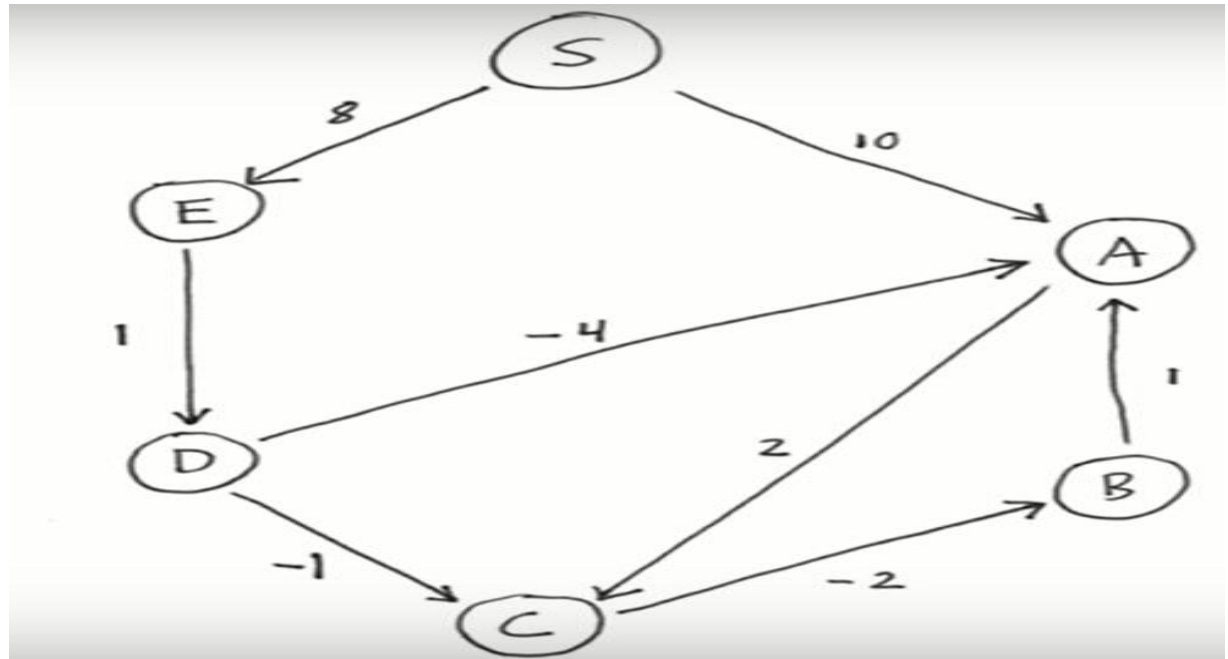
i.e. a set of nodes and lines(edges) connecting the nodes

**Routers –
represented by nodes**

**Networks-
represented by link
connecting two nodes**



A graph for Bellman-Ford algorithm



The algorithm is based on the fact that

if all neighbours of node i know the shortest distance to node j ,

then the shortest distance between node i and j can be found by adding the distance between node i and each neighbour to the neighbour's shortest distance to node j and then select the minimum

The fact behind Bellman-Ford algorithm

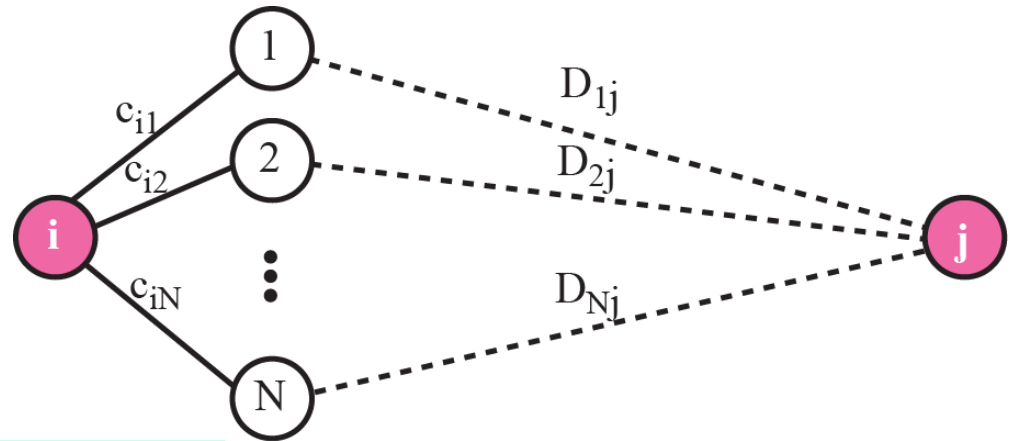
$$D_{ij} = \text{minimum } \{(c_{i1} + D_{1j}), (c_{i2} + D_{2j}), \dots, (c_{iN} + D_{Nj})\}$$

Legend

D_{ij} Shortest distance between i and j

c_{ij} Cost between i and j

N Number of nodes



Relaxing Edge

Relaxing an edge $\text{relax}(u, v)$ means to find shorter path to reach v when considering edge $u \rightarrow v$

$\text{relax}(u, v)$

if $d[v] > d[u] + w(u, v)$ then

$d[v] = d[u] + w(u, v)$

$p[v] = u$

create a shortest distance table (vector) for each node using the following steps

1. The shortest distance and the cost between a node and itself is initialized to 0.
2. The shortest distance between a node and any other node is set to infinity. The cost between a node and any other node should be given (can be infinity if the nodes are not connected).
3. The algorithm repeat until there is no more change in the shortest distance vector.

Table 11.1 *Bellman-Ford Algorithm*

```
1 Bellman_Ford ( )
2 {
3     // Initialization
4     for (i = 1 to N; for j = 1 to N)
5     {
6         if (i == j)   $D_{ij} = 0$      $c_{ij} = 0$ 
7         else           $D_{ij} = \infty$      $c_{ij} = \text{cost between } i \text{ and } j$ 
8     }
9     // Updating
10    repeat
11    {
12        for (i = 1 to N; for j = 1 to N)
13        {
14             $D_{ij} \leftarrow \text{minimum } [(c_{i1} + D_{1j}) \dots (c_{iN} + D_{Nj})]$ 
15        } // end for
16    } until (there was no change in previous iteration)
17 } // end Bellman-Ford
```

DISTANCE VECTOR ROUTING

If we want to use the algorithm for creating the routing table for routers in an AS, we need to change the algorithm:

1. In distance vector routing, the cost is normally hop counts (how many networks are passed before reaching the destination). So the cost between any two neighbors is set to 1.
2. Each router needs to update its routing table *asynchronously*, whenever it has received some information from its neighbors.
3. Processing is *distributive*. After a router has updated its routing table, it should send the result to its neighbors so that they can also update their routing table.
4. Each router should keep at least three pieces of information for each route: **destination network, the cost, and the next hop**. The whole routing table is referred as *Table*, the row *i* in the table as *Table_i*, the three columns in row *i* as **Table_i.dest**, **Table_i.cost**, and **Table_i.next**.
5. The information about each route received from a neighbor is referred as *R* (record), which has only two pieces of information: **R.dest** and **R.cost**. The next hop is not included in the received record because it is the source address of the sender.

Table 11.2 *Distance Vector Algorithm Used by Each Router*

```
1 Distance_Vector_Algorithm ( )
2 {
3     // At startup
4     for (i = 1 to N)           // N is number of ports
5     {
6         Tablei.dest = address of the attached network
7         Tablei.cost = 1
8         Tablei.next = —        // Means at home
9         Send a record R about each row to each neighbor
10    } // end for loop
11
12    // Updating
13    repeat (forever)
14    {
15        Wait for arrival of a record R from a neighbor
16        Update (R, T)           // Call update module
17        for (i = 1 to N)         // N is the current table size
```

Table 11.2 *Distance Vector Algorithm Used by Each Router (continued)*

```
18      {
19          Send a record R about each row to each neighbor
20      }
21  } // end repeat
22
23 } // end Distance_Vector
24 Update (R, T) // Update module
25 {
26     Search T for a destination matching the one in R
27     if (destination is found in row i)
28     {
29         if (R.cost + 1 < Ti.cost or R.next == Ti.next)
30         {
31             Ti.cost = R.cost + 1
32             Ti.next = Address of sending router
33         }
```

Address of the sender of *R*



```
34     else discard the record      // No change is needed
35 }
36 else
37     // Insert the new router
38 {
39      $T_{N+1}.dest = R.dest$ 
40      $T_{N+1}.cost = R.cost + 1$ 
41      $T_{N+1}.next = \text{Address of sending router}$ 
42     Sort the table according to destination address
43 }
44 } // end of Update module
```

Table 11.2 *Distance Vector Algorithm Used by Each Router (continued)*

Updating Routing Table

- If the next-node entry is different
 - The receiving node chooses the row with the smaller cost
 - If there is a tie, the old one is kept
- If the next-node entry is the same
(i.e. the sender of the new row is the provider of the old entry)
 - The receiving node chooses the new row, even though the new value is infinity.

When to Share

■ Periodic Update

- A node sends its routing table, normally 30 seconds, in a periodic update

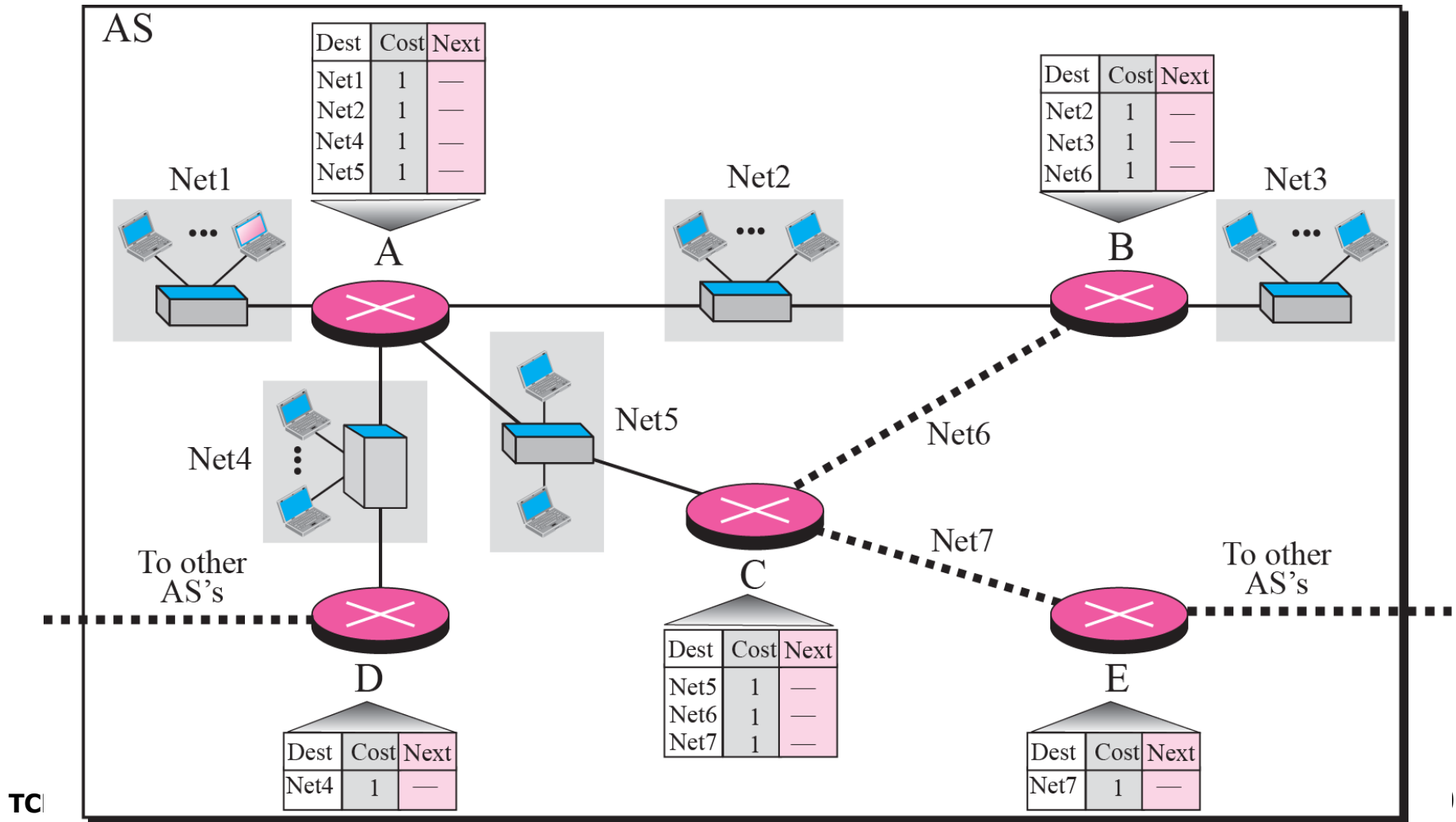
■ Triggered Update

- A node sends its routing table to its neighbors any time when there is a change in its routing table
 - 1. After updating its routing table, or
 - 2. Detects some failure in the neighboring links

Example: Figure shows the initial routing table for an AS

Note

- the figure does not mean that all routing tables have been created at the same time;
- each router creates its own routing table when it is booted.



Assume router A sends four records to its neighbors, routers B, D, and C. Figure shows the changes in B's routing table when it receives these records..



Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net6	1	—

After receiving record 1

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net6	1	—

After receiving record 2

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net6	1	—

After receiving record 3

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net5	2	A
Net6	1	—

After receiving record 4

Assume router A sends four records to its neighbors, routers B, D, and C. Figure shows the changes in B's routing table when it receives these records..



How router A is updated if this send to A from B

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net6	1	—

After receiving
record 1

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net6	1	—

After receiving
record 2

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net6	1	—

After receiving
record 3

Routing Table B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net5	2	A
Net6	1	—

After receiving
record 4

TCNet7	1	—	quite
--------	---	---	-------

A

Dest	Cost	Next
Net1	1	—
Net2	1	—
Net3	2	B
Net4	1	—
Net5	1	—
Net6	2	C
Net7	2	C

B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net5	2	A
Net6	1	—
Net7	2	C

C

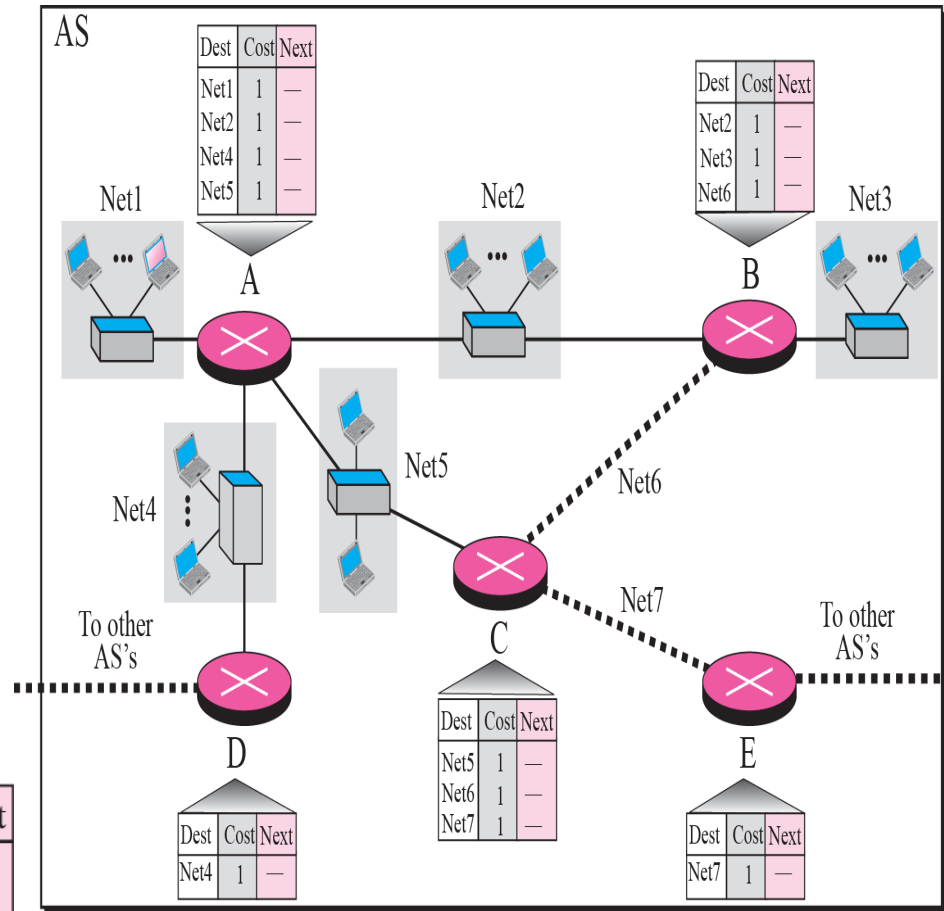
Dest	Cost	Next
Net1	2	A
Net2	2	A
Net3	2	B
Net4	2	A
Net5	1	—
Net6	1	—
Net7	1	—

D

Dest	Cost	Next
Net1	2	A
Net2	2	A
Net3	3	A
Net4	1	—
Net5	1	A
Net6	3	A
Net7	3	A

E

Dest	Cost	Next
Net1	3	C
Net2	3	C
Net3	3	C
Net4	3	C
Net5	2	C
Net6	2	C
Net7	1	—



final routing tables for router D

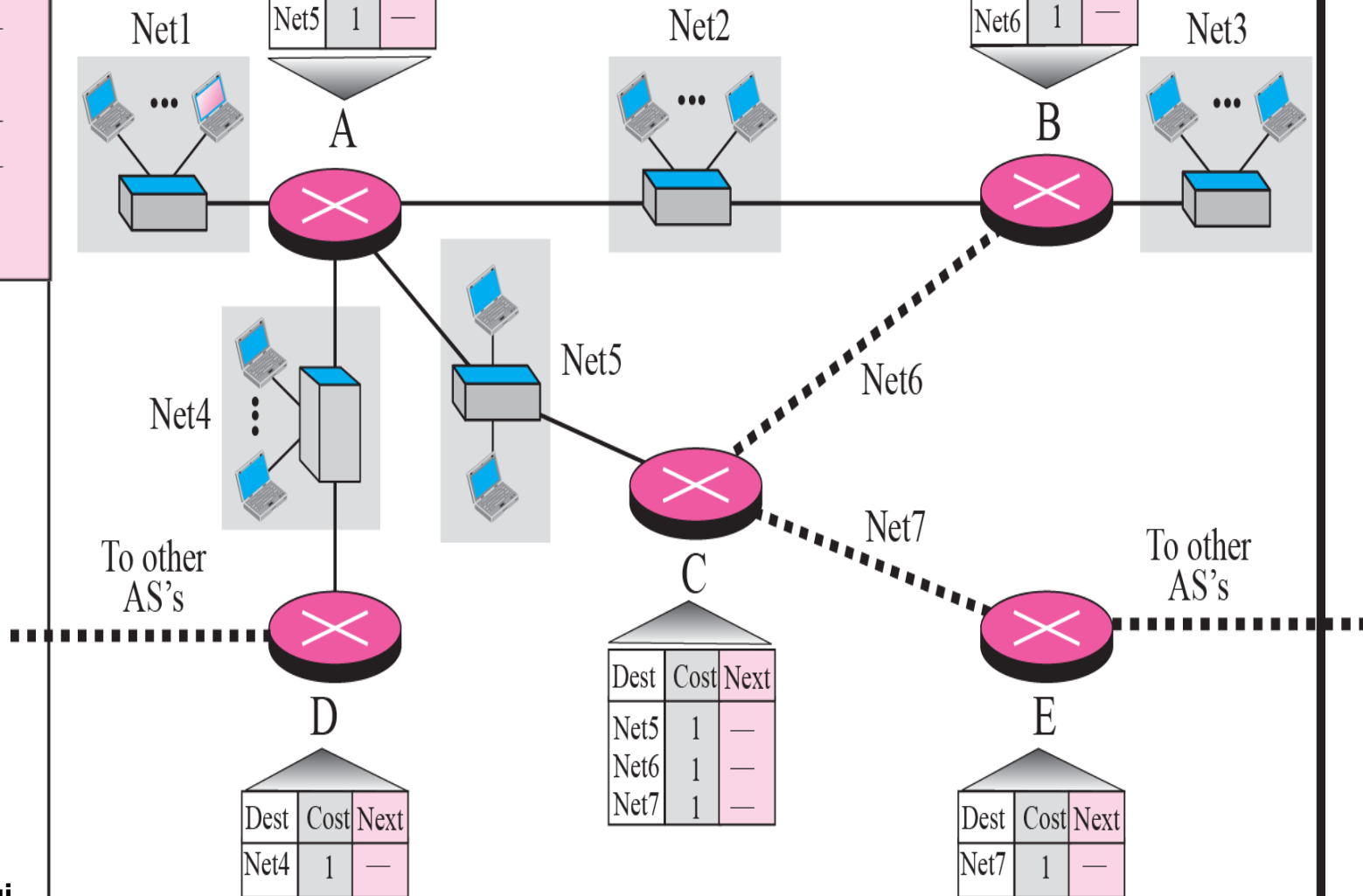
A

Dest	Cost	Next
Net1	1	—
Net2	1	—
Net3	2	B
Net4	1	—
Net5	1	—
Net6	2	C
Net7	2	C

AS

Dest	Cost	Next
Net1	1	—
Net2	1	—
Net4	1	—
Net5	1	—

Dest	Cost	Next
Net2	1	—
Net3	1	—
Net6	1	—



final routing tables for router D

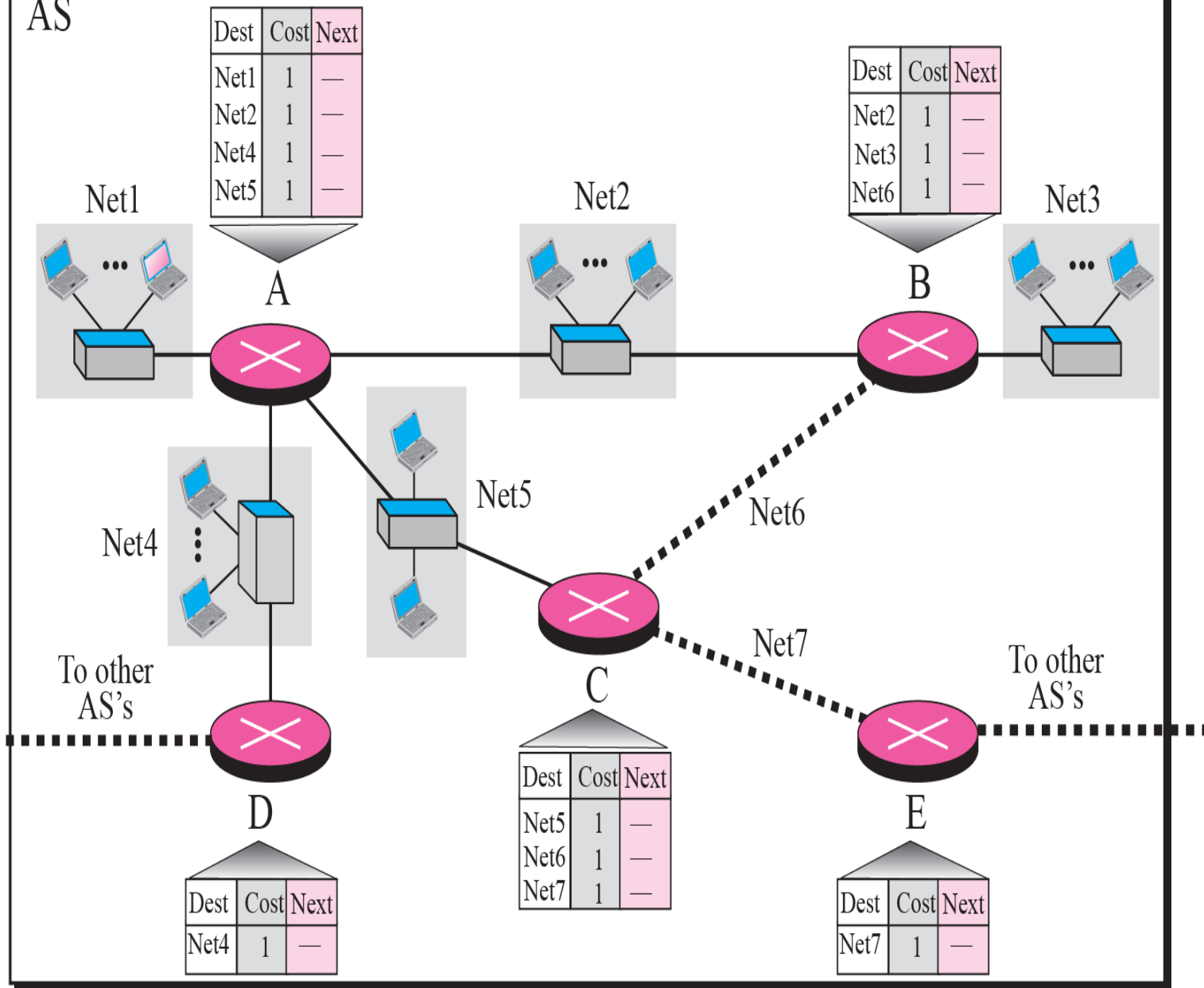
B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net5	2	A
Net6	1	—
Net7	2	C

D

Dest	Cost	Next
Net1	2	A
Net2	2	A
Net3	3	A
Net4	1	—
Net5	1	A
Net6	3	A
Net7	3	A

AS



Assume if table B is table for router A (*adjust Next-hop entries, change A entries to B*) and send the records to router D, update Table D

B

Dest	Cost	Next
Net1	2	A
Net2	1	—
Net3	1	—
Net4	2	A
Net5	2	A
Net6	1	—
Net7	2	C

D

Dest	Cost	Next
Net1	2	A
Net2	2	A
Net3	3	A
Net4	1	—
Net5	2	A
Net6	3	A
Net7	3	A

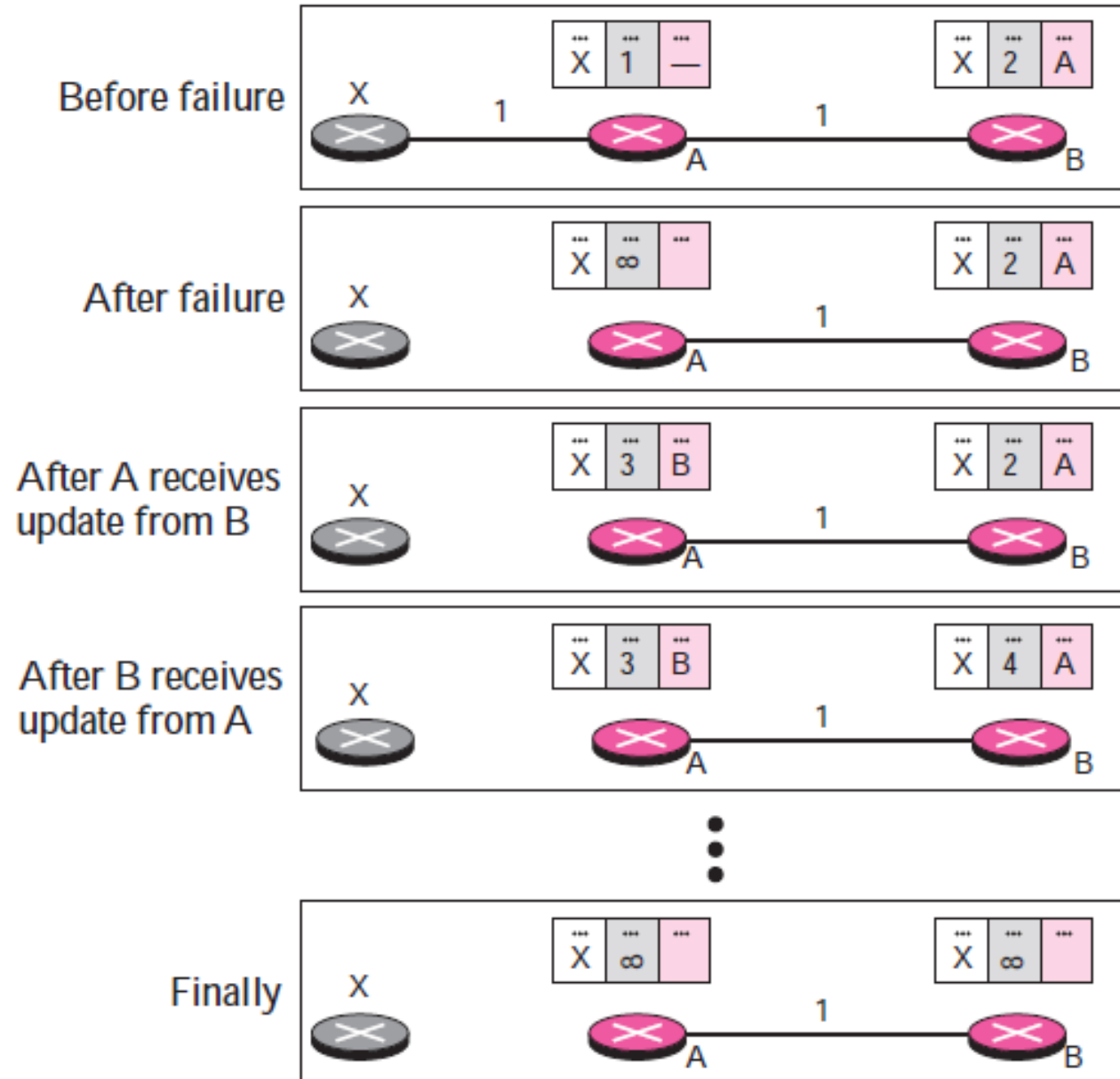
Count to Infinity

Problem with distance vector routing

- Good news (decrease in cost) propagates quickly
- Bad news (increase in cost) takes time.

It **takes several series** on updates before the *cost for a broken link is recorded as infinity* by all routers

Two Node Instability



Solutions to Two-Node Instability

■ Defining Infinity

- Most implementations define 16 as infinity
- Therefore, DVR cannot be implemented in a larger system

■ Split Horizon

- Instead of flooding the table through each interface, each node sends only part of its table through each interface
- E.g. node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A

■ Split Horizon and Poison Reverse

■ One drawback of Split Horizon

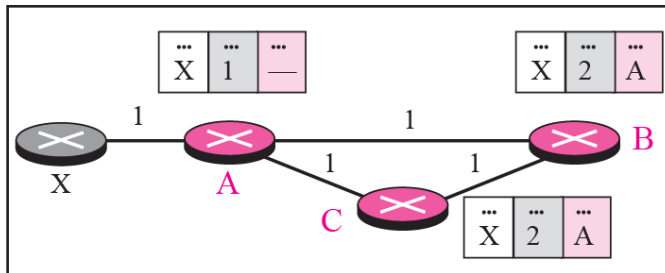
- Normally, the DV protocol uses a timer and if there is no news about a route, the node deletes the route from its table
- In the previous e.g., node A cannot guess that this is due to split horizon or because B has not received any news about X recently

■ Poison Reverse

- Node B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning

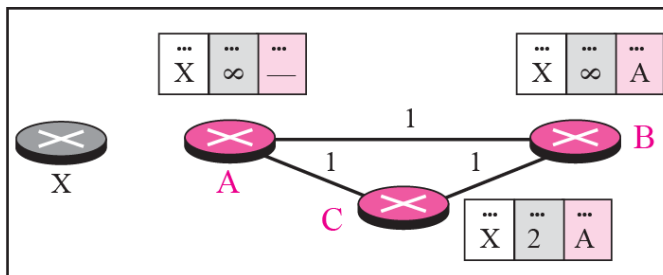
Three-node instability

Before failure

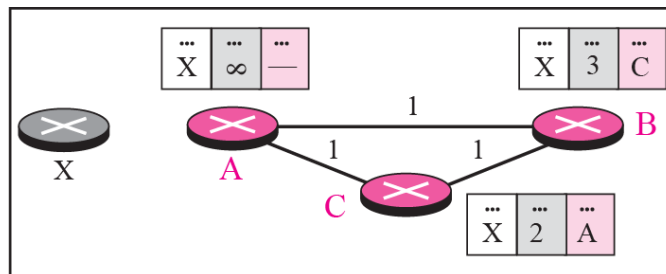


If the instability is btw three nodes, stability cannot be guaranteed

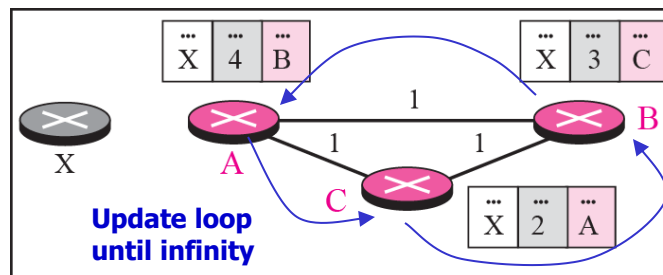
After A sends the route to B and C, but the packet to C is lost



After C sends the route to B



After B sends the route to A

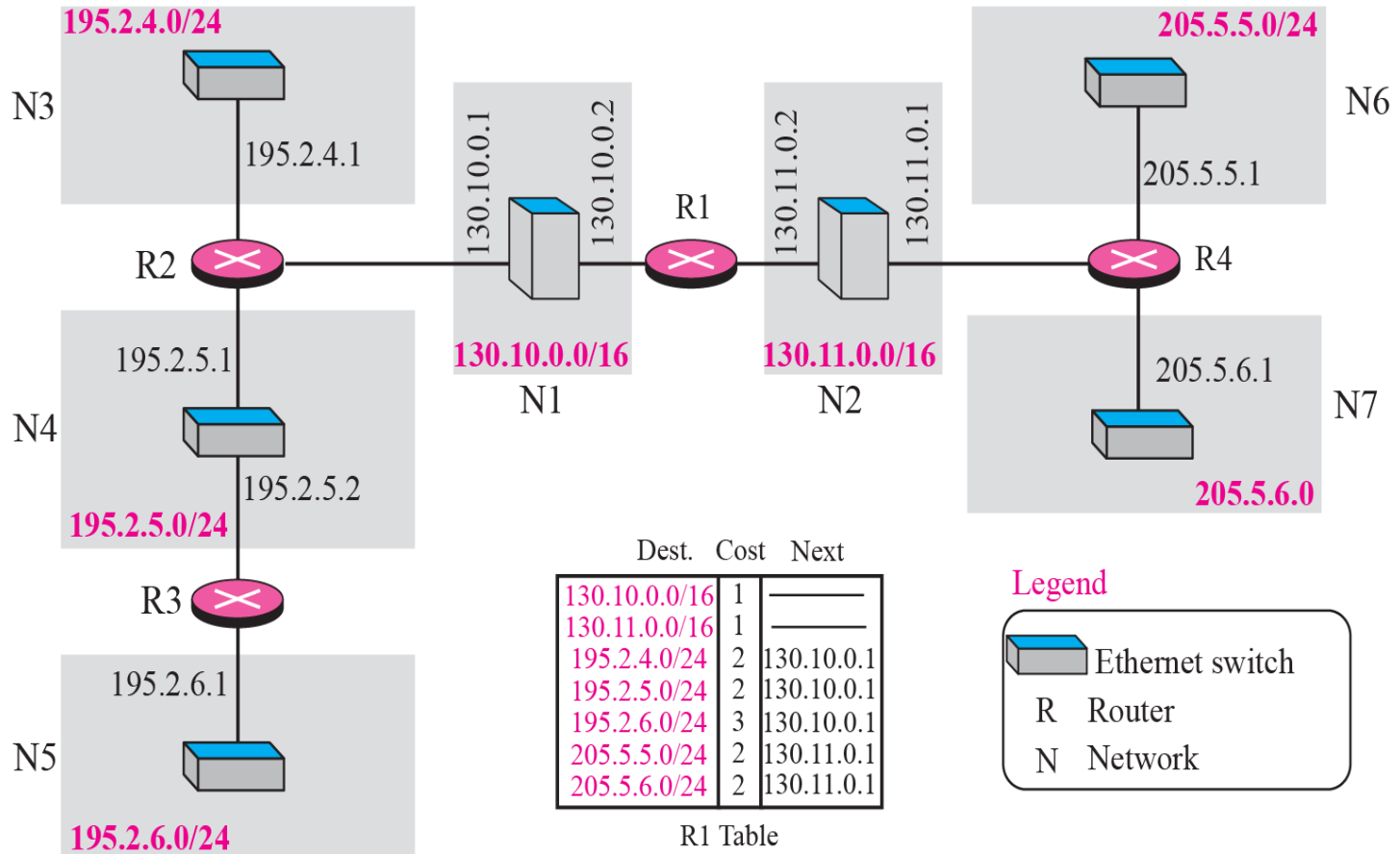


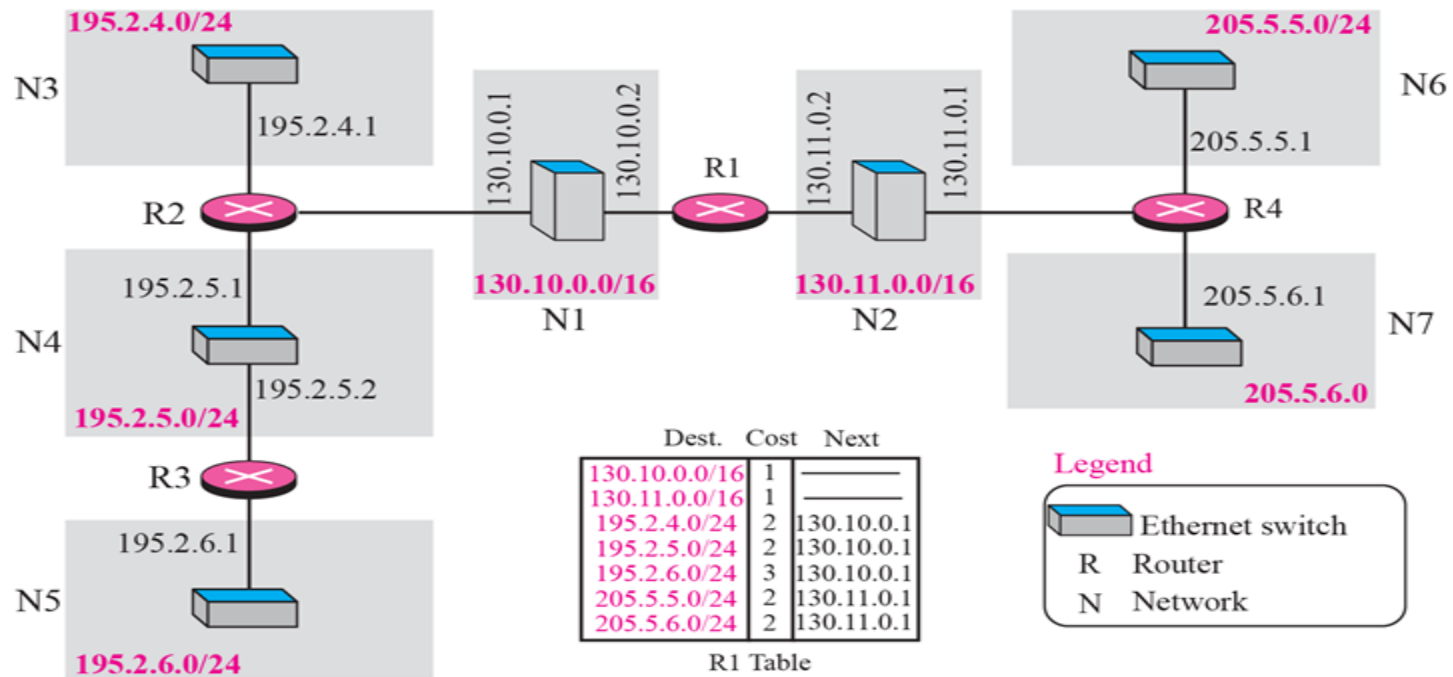
RIP- Routing Information Protocol

RIP is an intra-domain (interior) routing protocol used inside an autonomous system. RIP implements distance vector routing directly with some considerations.

1. In an autonomous system, we are dealing with routers and networks (links), what was described as a node.
2. The destination in a routing table is a network, which means the first column defines a network address.
3. The metric used by RIP is very simple; the distance is defined as the number of links (networks) that have to be used to reach the destination. For this reason, the metric in RIP is called a **hop count**.
4. Infinity is defined as 16, which means that any route in an autonomous system using RIP cannot have more than 15 hops.
5. The next node column defines the address of the router to which the packet is to be sent to reach its destination.

Example of a domain using RIP





Dest. Cost Next

130.10.0.0/16	1	—
130.11.0.0/16	2	130.10.0.2
195.2.4.0/24	1	—
195.2.5.0/24	1	—
195.2.6.0/24	2	195.2.5.2
205.5.5.0/24	3	130.10.0.2
205.5.6.0/24	3	130.10.0.2

R2 Table

Dest. Cost Next

130.10.0.0/16	2	195.2.5.1
130.11.0.0/16	3	195.2.5.1
195.2.4.0/24	2	195.2.5.1
195.2.5.0/24	1	—
195.2.6.0/24	1	—
205.5.5.0/24	4	195.2.5.1
205.5.6.0/24	4	195.2.5.1

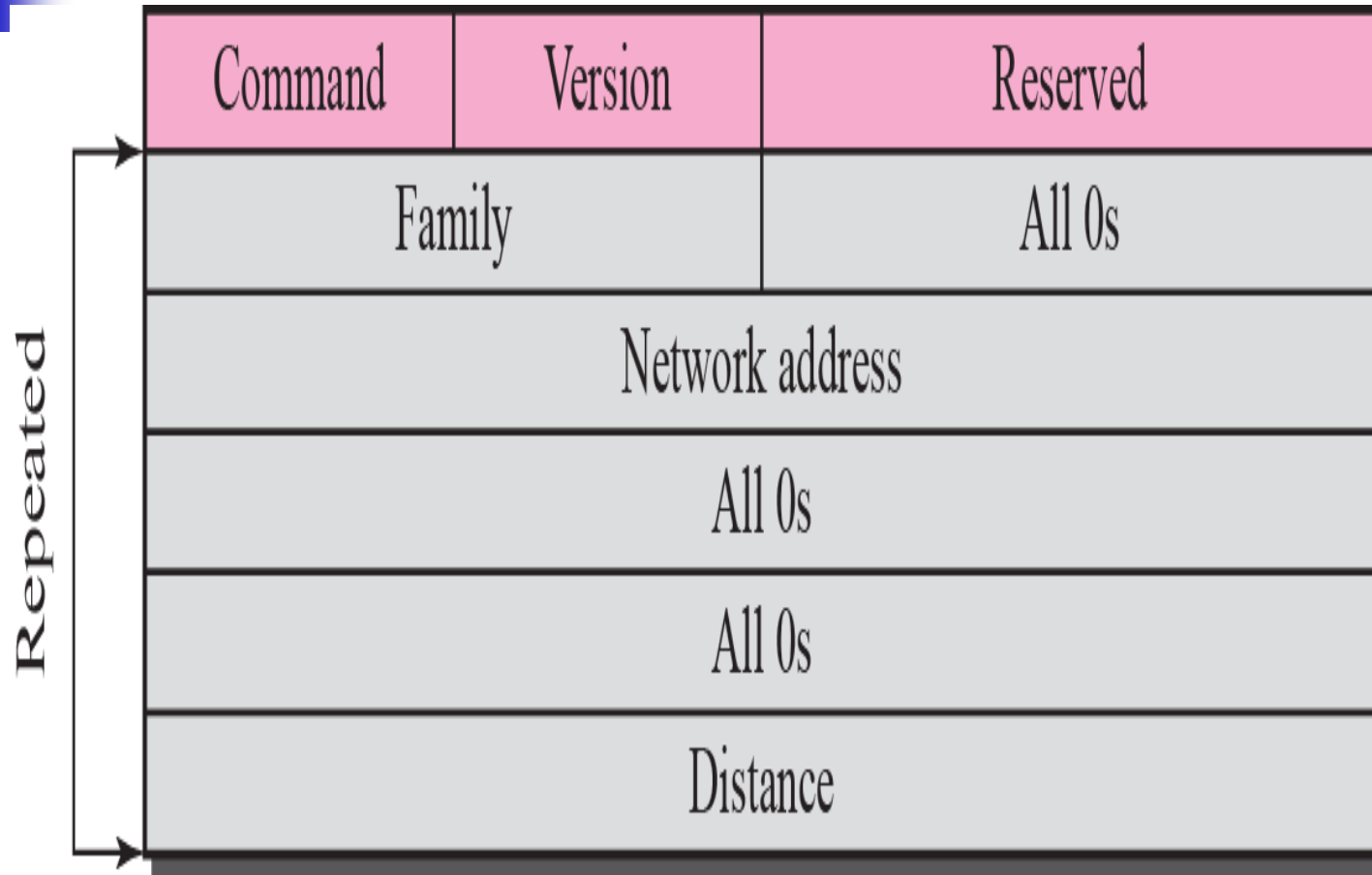
R3 Table

Dest. Cost Next

130.10.0.0/16	2	130.11.0.2
130.11.0.0/16	1	—
195.2.4.0/24	3	130.11.0.2
195.2.5.0/24	3	130.11.0.2
195.2.6.0/24	4	130.11.0.2
205.5.5.0/24	1	—
205.5.6.0/24	1	—

R4 Table

RIP message format



RIP Message Format

- Each message contains a command and a version number and can contain entries for up to **25 routes**.
- Each route entry includes an address family identifier, the IP address reachable by the route, and the hop count for the route.
- If a router must send an update with more than 25 entries, multiple RIP messages must be produced.

maximum RIP size

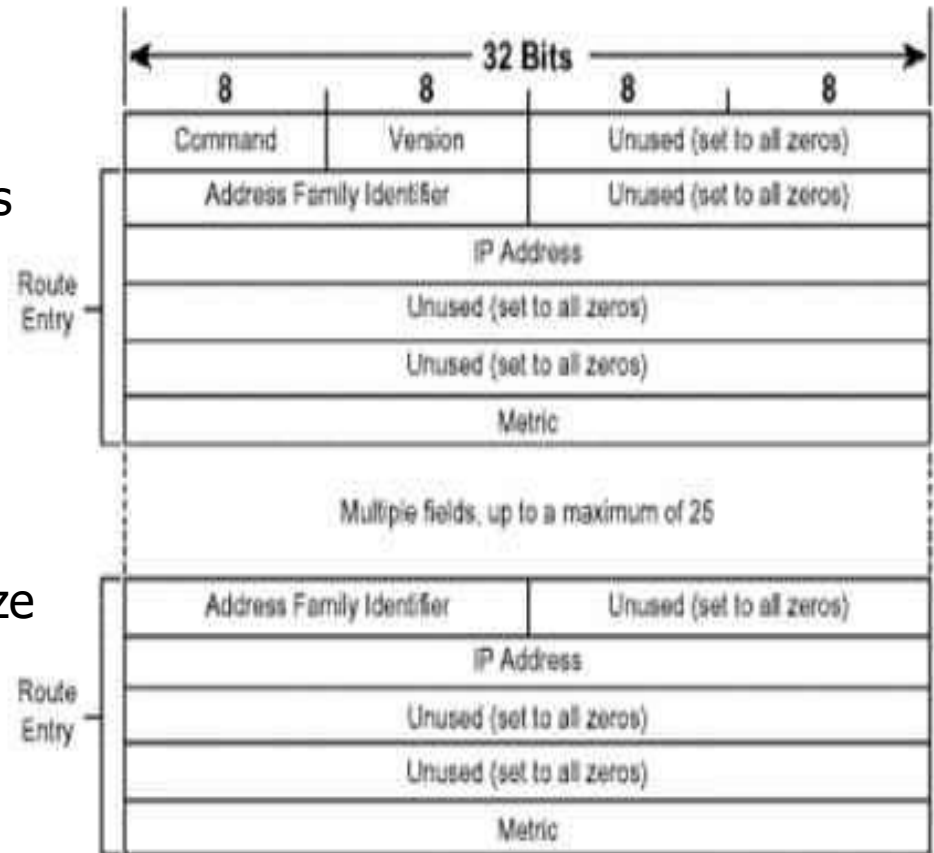
- initial portion of the message is 4 octets
- each route entry is 20 octets.

Therefore, the maximum message size is

$$4 + (25 \times 20) = 504 \text{ octets}$$

If we include an 8 byte UDP header

Then, maximum RIP datagram payload size
512 octets



RIP messages

■ Request

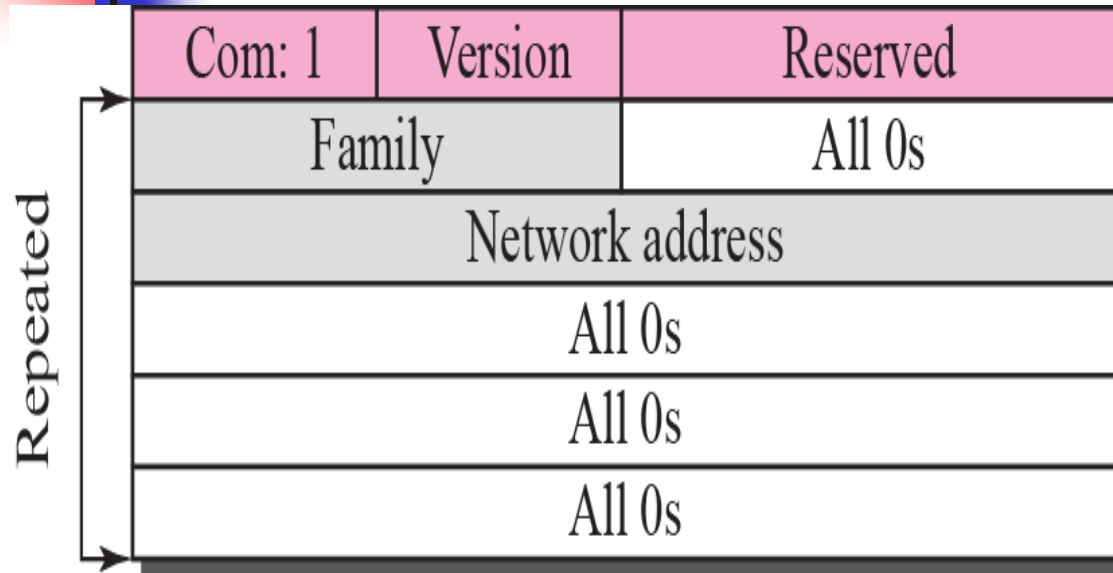
- A request message is sent by a router that has just come up or by a router that has some time-out entries
- A request can ask about specific entries or all entries

■ Response

A response can be either solicited or unsolicited

- solicited (as a reply to request)
- unsolicited (30s or when any change in the routing table)

Request messages

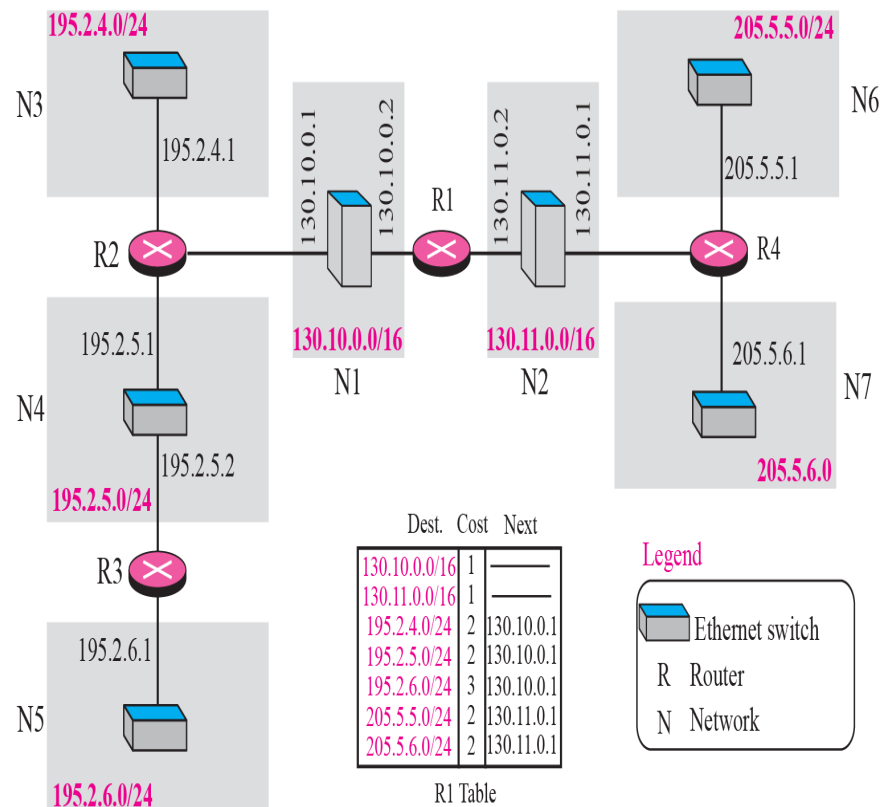


a. Request for some

Com: 1	Version	Reserved
Family		All 0s
All 0s		
All 0s		
All 0s		
All 0s		

b. Request for all

An update message sent from router R1 to router R2



Dest.	Cost	Next
130.10.0.0/16	1	—
130.11.0.0/16	1	—
195.2.4.0/24	2	130.10.0.1
195.2.5.0/24	2	130.10.0.1
195.2.6.0/24	3	130.10.0.1
205.5.5.0/24	2	130.11.0.1
205.5.6.0/24	2	130.11.0.1

R1 Table

Legend

Ethernet switch

Router

Network

Dest.	Cost	Next
130.10.0.0/16	1	—
130.11.0.0/16	2	130.10.0.2
195.2.4.0/24	1	—
195.2.5.0/24	1	—
195.2.6.0/24	2	195.2.5.2
205.5.5.0/24	3	130.10.0.2
205.5.6.0/24	3	130.10.0.2

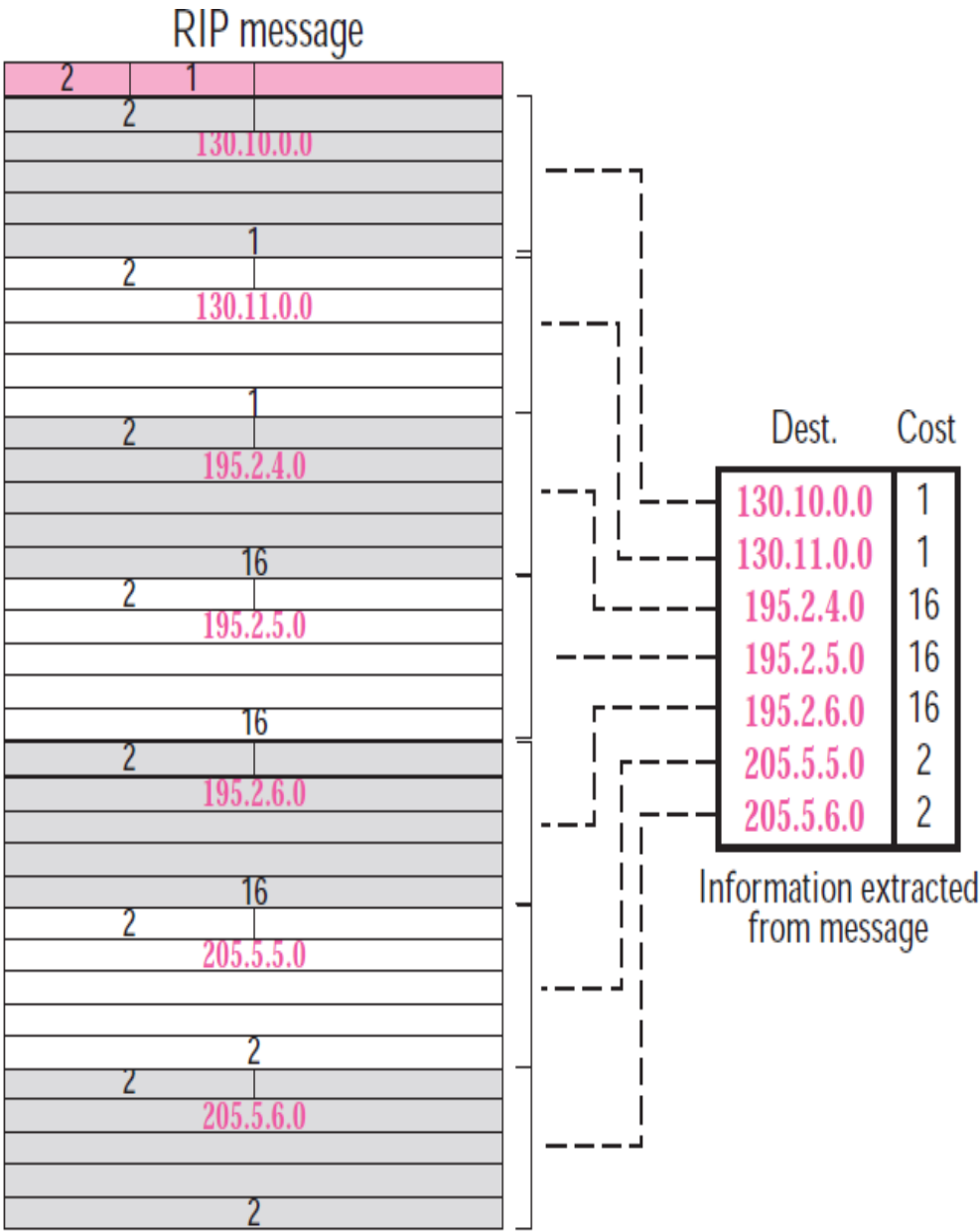
R2 Table

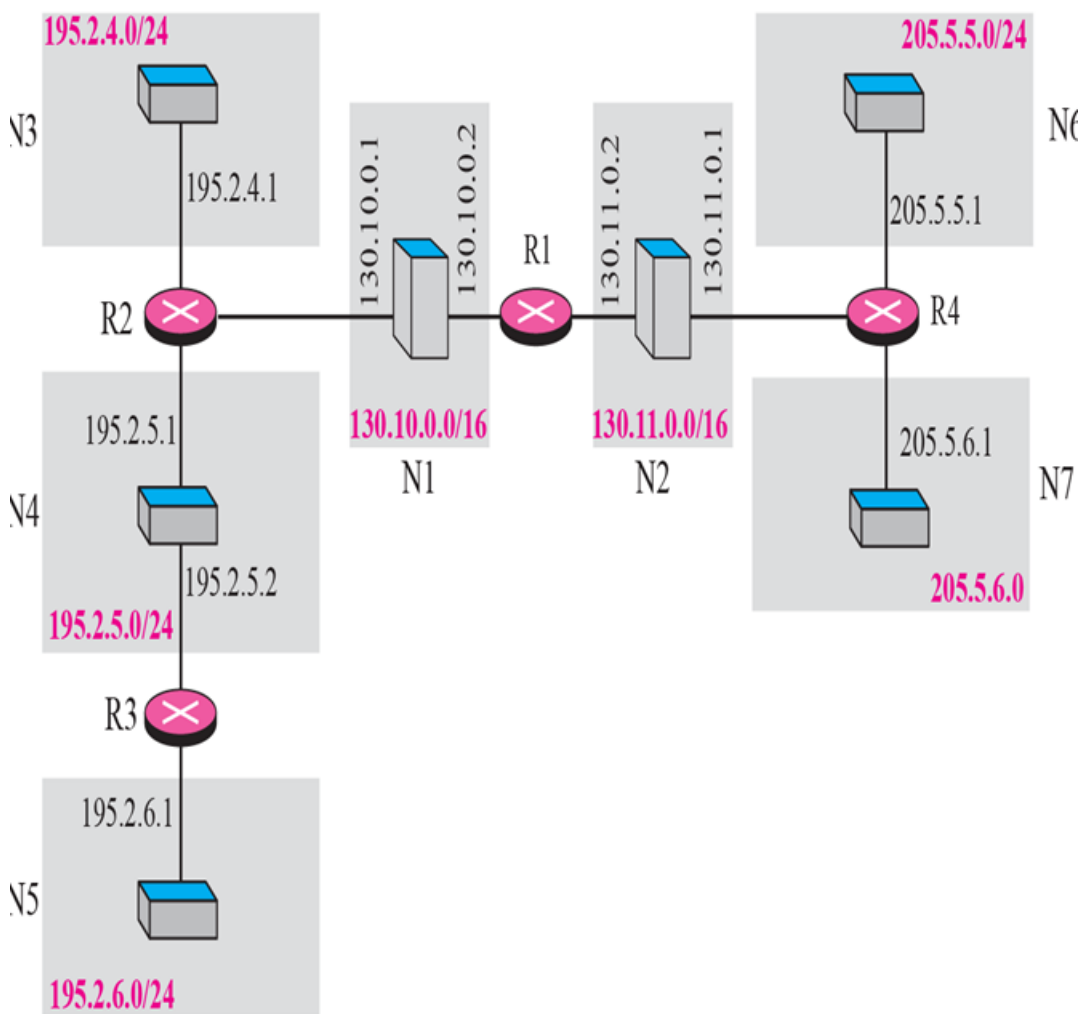
Dest.	Cost	Next
130.10.0.0/16	2	195.2.5.1
130.11.0.0/16	3	195.2.5.1
195.2.4.0/24	2	195.2.5.1
195.2.5.0/24	1	—
195.2.6.0/24	1	—
205.5.5.0/24	4	195.2.5.1
205.5.6.0/24	4	195.2.5.1

R3 Table

Dest.	Cost	Next
130.10.0.0/16	2	130.11.0.2
130.11.0.0/16	1	—
195.2.4.0/24	3	130.11.0.2
195.2.5.0/24	3	130.11.0.2
195.2.6.0/24	4	130.11.0.2
205.5.5.0/24	1	—
205.5.6.0/24	1	—

R4 Table

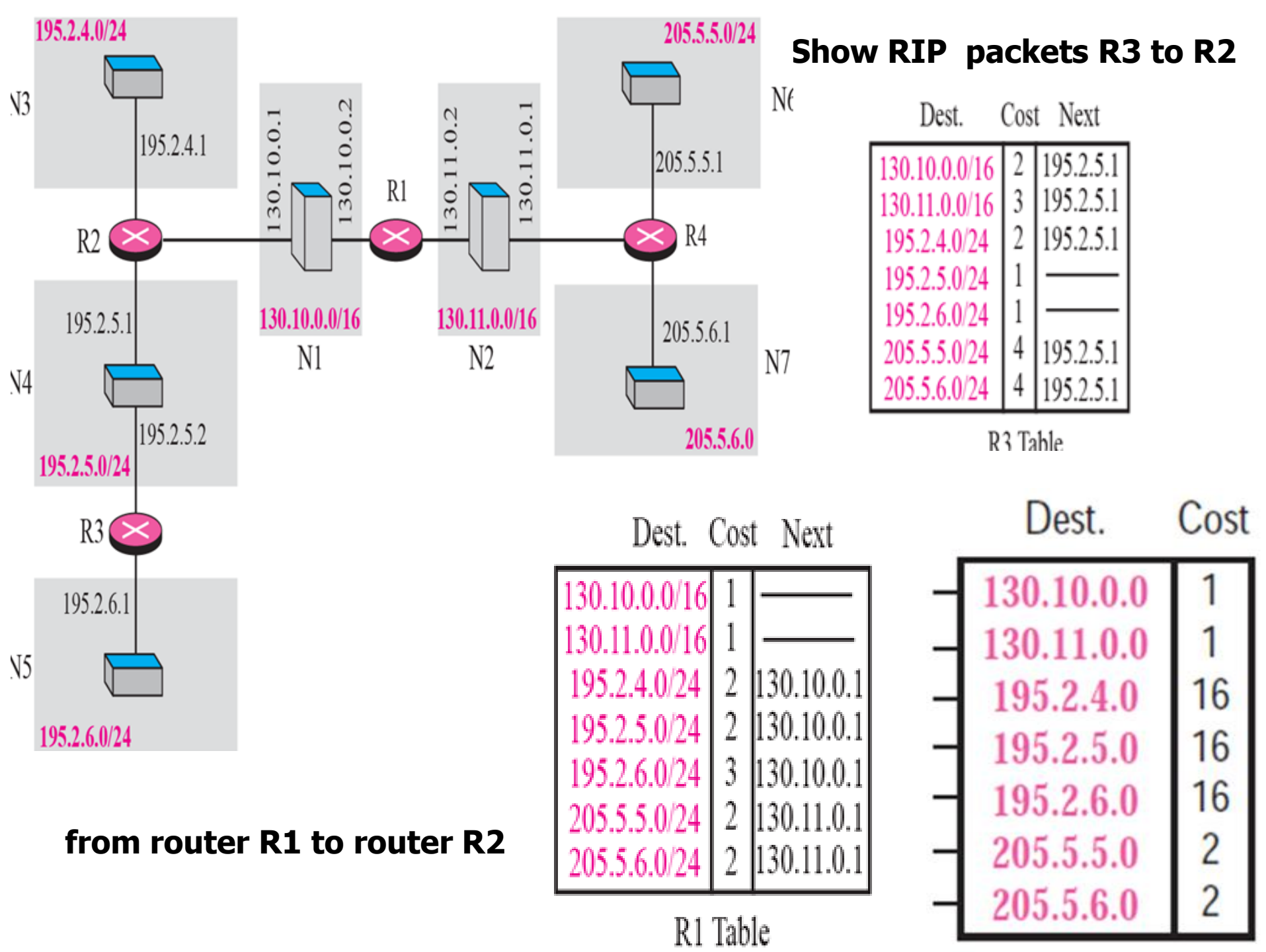




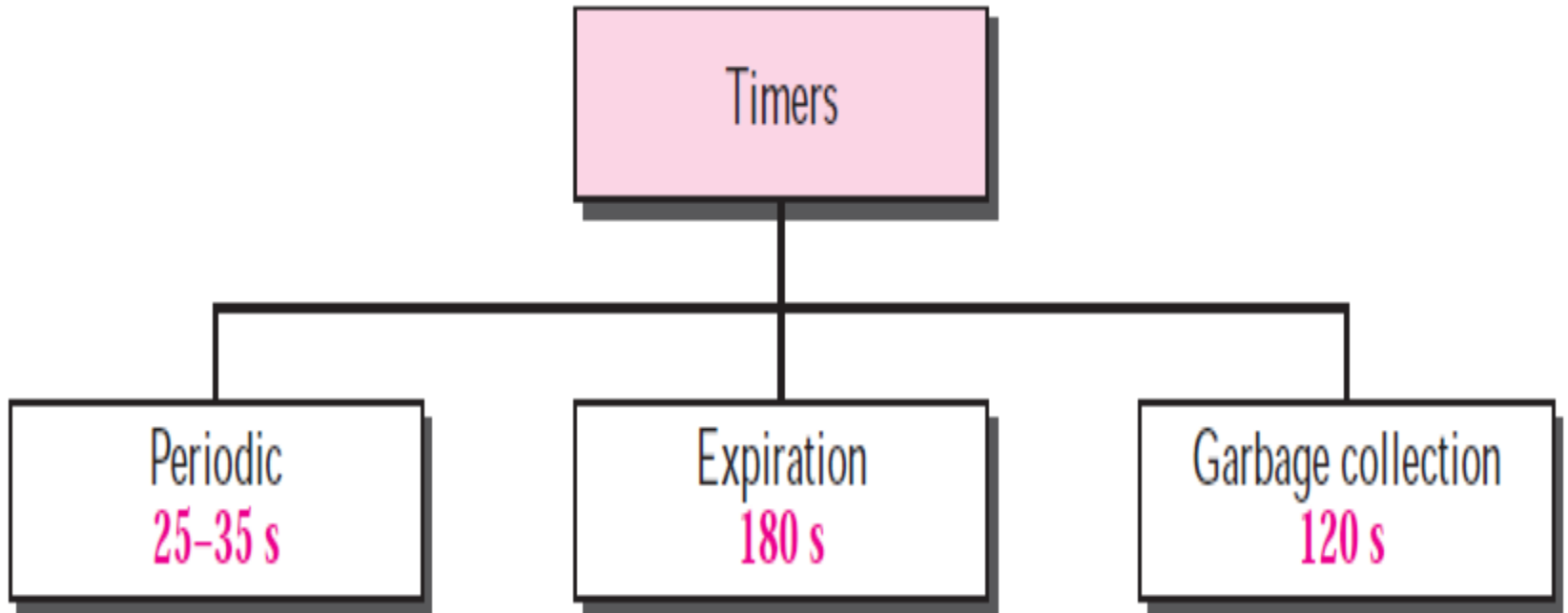
Dest.	Cost
130.10.0.0	1
130.11.0.0	1
195.2.4.0	16
195.2.5.0	16
195.2.6.0	16
205.5.5.0	2
205.5.6.0	2

Dest.	Cost	Next
130.10.0.0/16	1	—
130.11.0.0/16	2	130.10.0.2
195.2.4.0/24	1	—
195.2.5.0/24	1	—
195.2.6.0/24	2	195.2.5.2
205.5.5.0/24	3	130.10.0.2
205.5.6.0/24	3	130.10.0.2

R2 Table



Timers in RIP



RIP Timers

■ Periodic timer

- It controls the advertising of regular update message (25 ~ 35 sec)

■ Expiration timer

- It governs the validity of a route (180 sec)
- The route is considered expired and the hop count of the route is set to 16

■ Garbage collection timer

- A invalid route is not purged from the routing table until this timer expires (120 sec)
- It continues to advertise the route with metric value 16

Example 11.5

A routing table has 20 entries. It does not receive information about five routes for 200 s.

How many timers are running at this time?



Example 11.5

A routing table has 20 entries. It does not receive information about five routes for 200 s.

How many timers are running at this time?

Solution

The 21 timers are listed below:

Periodic timer: 1

Expiration timer: $20 - 5 = 15$

Garbage collection timer: 5

**A router has 300 entries at any instance,
how many expiration timers are active at this
instant**

Ques A router using RIP has the routing table shown below

<i>Destination</i>	<i>Cost</i>	<i>Next Router</i>
Net1	4	B
Net2	2	C
Net3	1	F
Net4	5	G

This router receives a RIP message with four records from router C as shown below:

(Net1, 2), (Net2, 2), (Net3, 3), (Net4, 7)

Show the updated routing table.

Ques A router using RIP has the routing table shown below

<i>Destination</i>	<i>Cost</i>	<i>Next Router</i>
Net1	4	B
Net2	2	C
Net3	1	F
Net4	5	G

four records from router C as shown below:
(Net1, 2), (Net2, 2), (Net3, 3), (Net4, 7)

updated routing table.

Dest	Cost	Next
Net 1	3	C
Net 2	3	C
Net 3	1	F
Net 4	5	G

Holddown timer

It is used by DVR protocols (such as **RIP**) to prevent routing loops

This feature prevents a router from learning new information about a failed route until the timer expires.

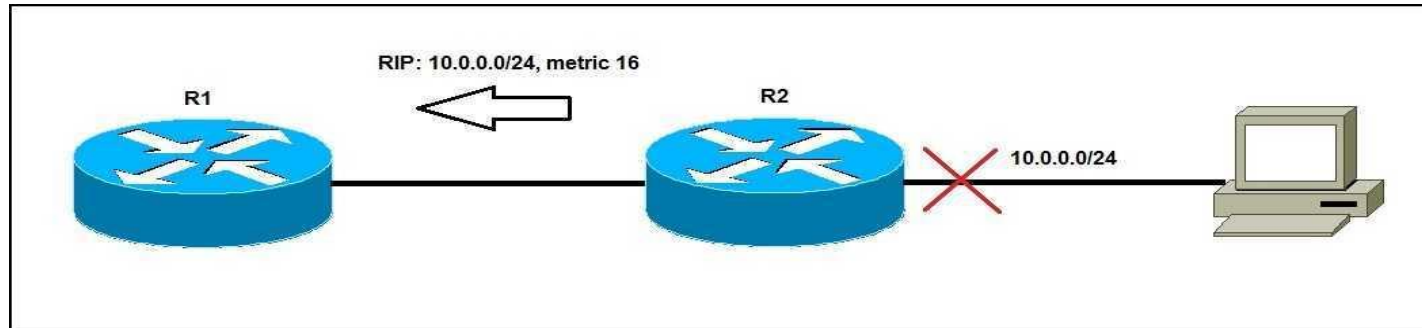
Here is how it works:

1. a router receives an information that a route is unreachable and starts the hold-down timer.
2. until the holddown timer expires (**180 seconds** by default in **RIP**), the router will discard any routing updates that indicate the route is reachable.
3. only routing updates about the failed route that will be processed are the ones sent by the same router that originally advertised the route.

What is a hold timer?

If a router receives a notification that a specific path or network has gone offline, the router starts a timer and discards all the updates it gets from the neighboring routers.

Consider the following example: We have a network of two routers.

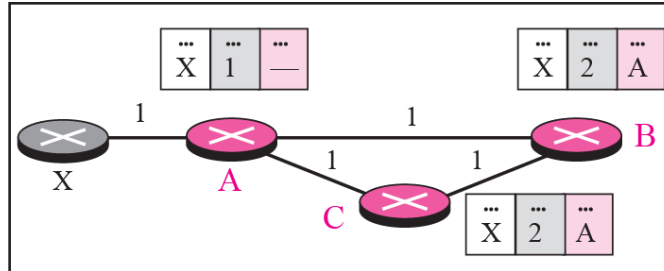


Both routers are running RIP and R2 has advertised the **10.0.0.0/24** network to R1. Consider what happens if the network fails:

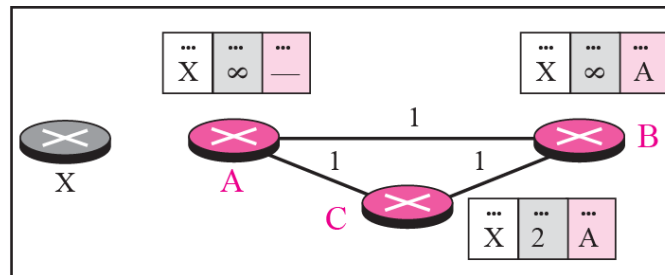
1. R2 advertises the **10.0.0.0/24** network with the infinitive metric (**16**) to R1, indicating that the network is no longer accessible.
2. R1 receives the routing update, marks the route as unreachable, and starts the **holddown timer**.
3. During the holddown period, R1 will not process any routing update about that route received from other routers. Only updates from R2 will be processed:

Three-node instability

Before failure



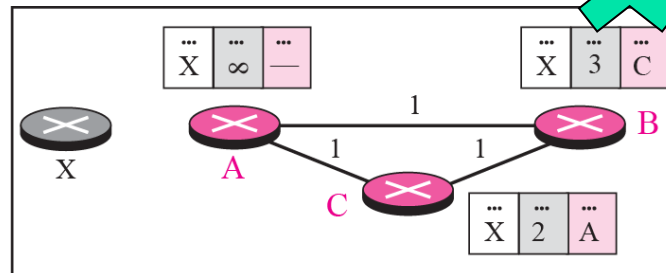
After A sends the route to B and C, but the packet to C is lost



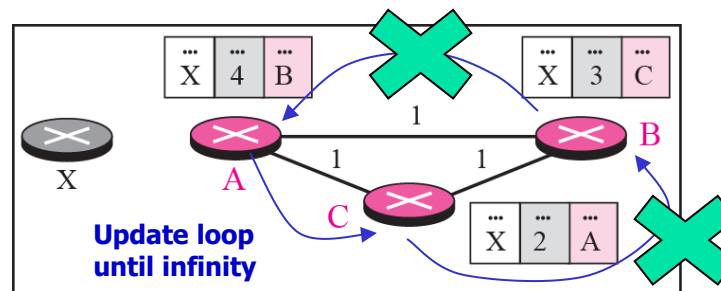
a router receives an information that a route is unreachable and starts the hold-down timer.

until the holddown timer expires (**180 seconds**), the router will discard any routing updates that indicate the route is reachable.

After C sends the route to B



After B sends the route to A



Lecture videos on the topic can be found at

TCP-IP Routing Basics

<https://youtu.be/gUdK3LyTmPo>

Distance Vector Routing - DVR - Understanding its features, Algorithms with an example

<https://youtu.be/eo9T6Uc6qAo>

Count to Infinity: Node Instability Problem in Routing

<https://youtu.be/Q1vqMoyEyAI>

RIP : Routing Information Protocol

https://youtu.be/WB_qNFktoDQ

RIP Timers

<https://youtu.be/HPNKwcCRqIQ>

Simulating Routing Information Protocol using Cisco packet Tracer

<https://youtu.be/g2uaB0DNI9k>